

Cybersecurity

PowerShell Scripting

Lesson 7.2



Learning Objectives

By the end of today's class, you will be able to:

- 1 Use basic PowerShell cmdlets to navigate Windows and manage directories and files.
- 2 Use PowerShell pipelines to retrieve Windows system event logs.
- 3 Combine various shell-scripting concepts such as cmdlets, parameters, conditions, and importing files with defined data structures.



Welcome



Recap

In the last class, we discussed:

1 Auditing processes

2 Using CMD to create files

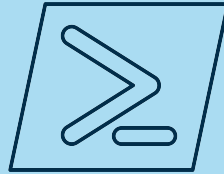
3 Windows Users

4 Auditing unwanted apps

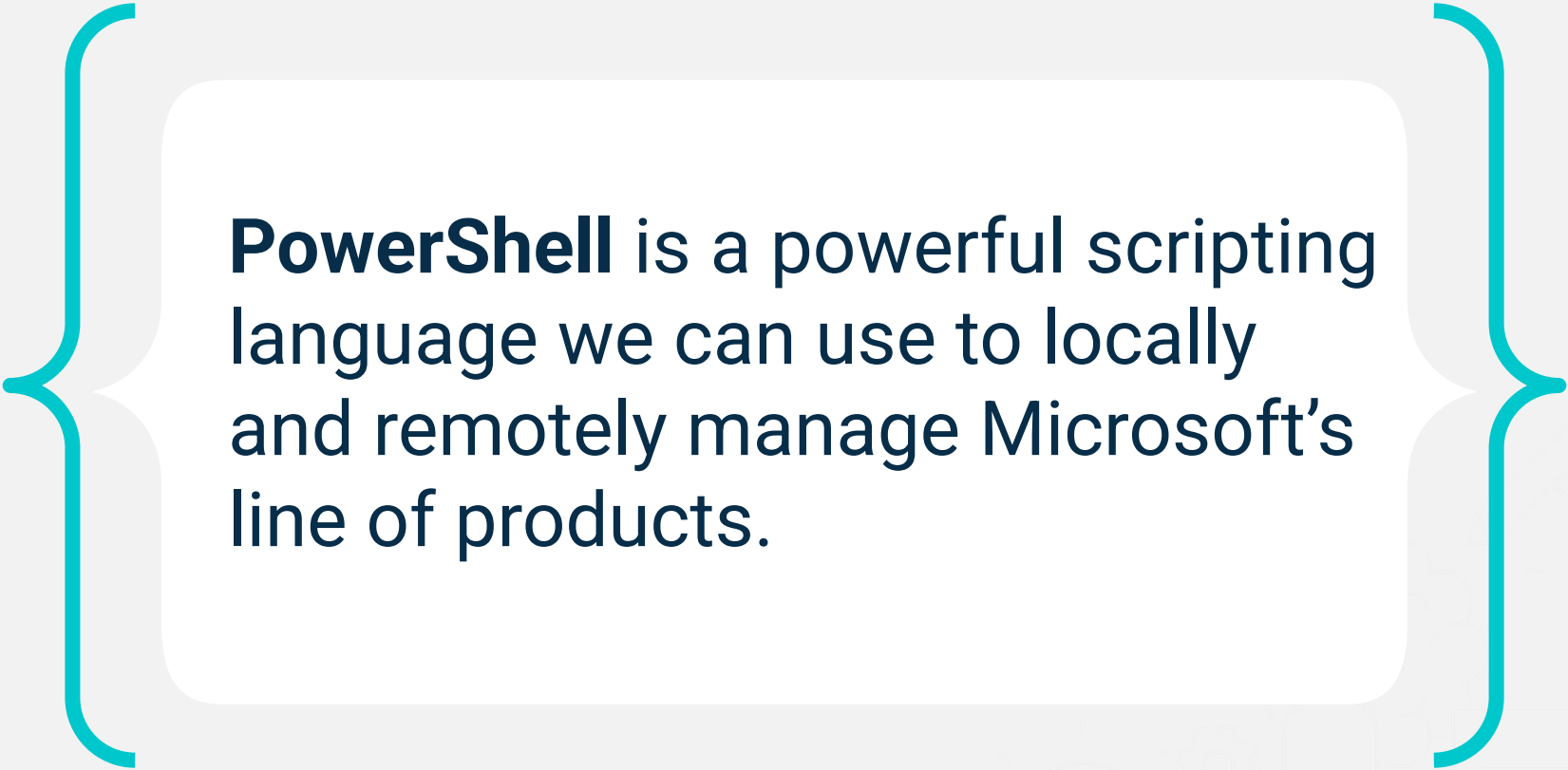
5 Enumerating local policies

6 Creating users with policies

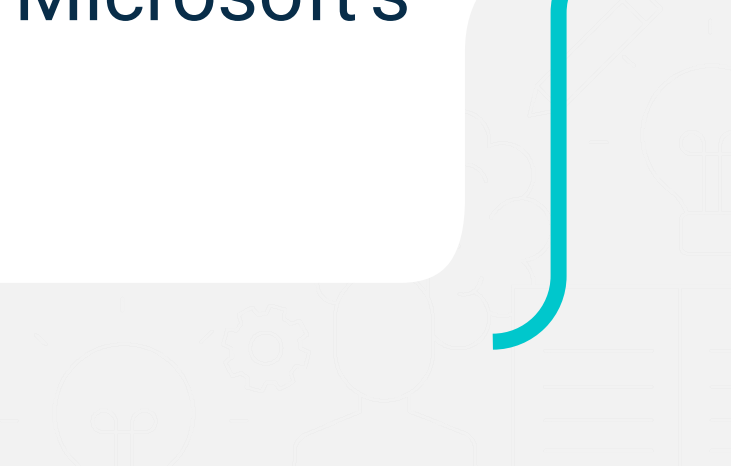
7 Task Scheduler



What is **PowerShell**?

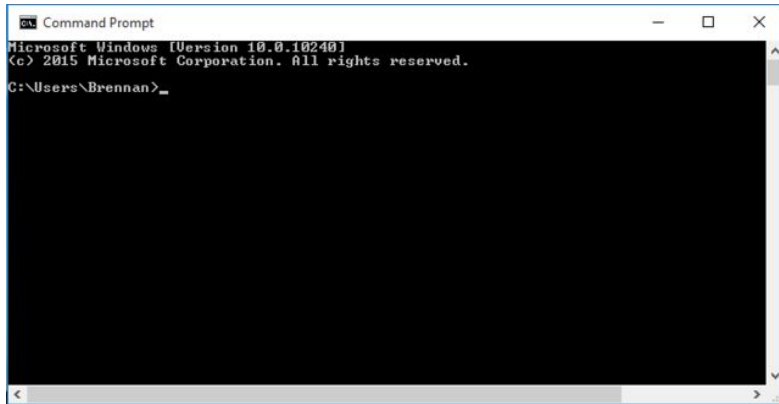


PowerShell is a powerful scripting language we can use to locally and remotely manage Microsoft's line of products.



CMD

Commands are short and simple,
designed for repeatable tasks.



PowerShell

Powerful language used to
execute, automate, and customize
tasks.





PowerShell in System Administration

PowerShell can be used to manage everything in a Microsoft environment.



Windows Server

Windows Server: Microsoft's central server for domain and networking services.



Windows 10

Windows 10: Microsoft's personal and professional computer operating system, the operating system used by most people and within many large enterprise environments.



Microsoft 365

Office365: Microsoft's line of subscription office services, such as Microsoft Word or Excel.



Azure: Microsoft's cloud services, such as virtual private clouds and cloud-based VMs.



PowerShell in Cybersecurity

Defensive Cybersecurity

- Manage and audit logs
- Harden hosts and servers
- Extended module and script functionality



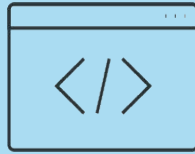


PowerShell in Cybersecurity

Offensive Cybersecurity

- “Living off the land” tool
- Enumerate systems on a network
- Access other network machines





PowerShell vs. CMD

PowerShell vs. CMD

CMD's functionality is limited.

CMD output is only available in simple text format.

CMD command flags can be ambiguous, confusing, and specific to each command:

For unsupported file formats, we need to edit the output with meticulous character replacing.

/s flags:

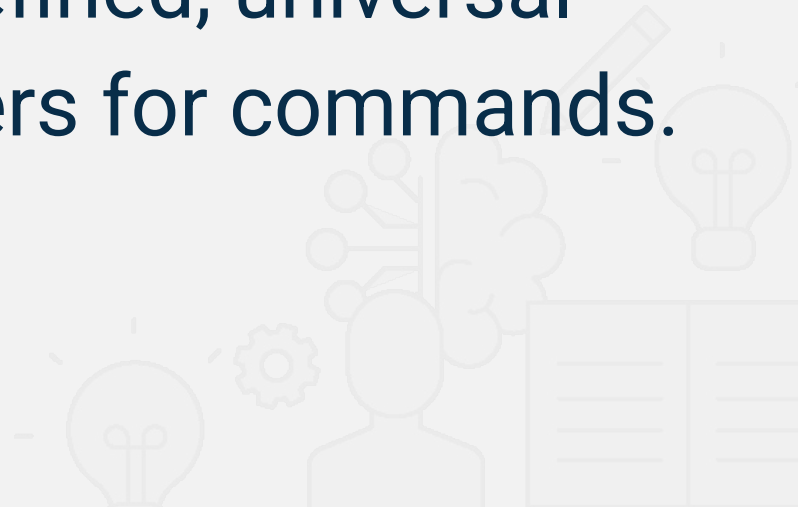
- `shutdown /s` shuts down a computer.
- `freedisk /s` specifies the name of an IP or remote computer to check disk space.

/d flags:

- `shutdown /d` specifies a reason for shut down.
- `freedisk /d` specifies which disk drive to check.



PowerShell provides
clearly defined, universal
parameters for commands.



PowerShell vs. CMD

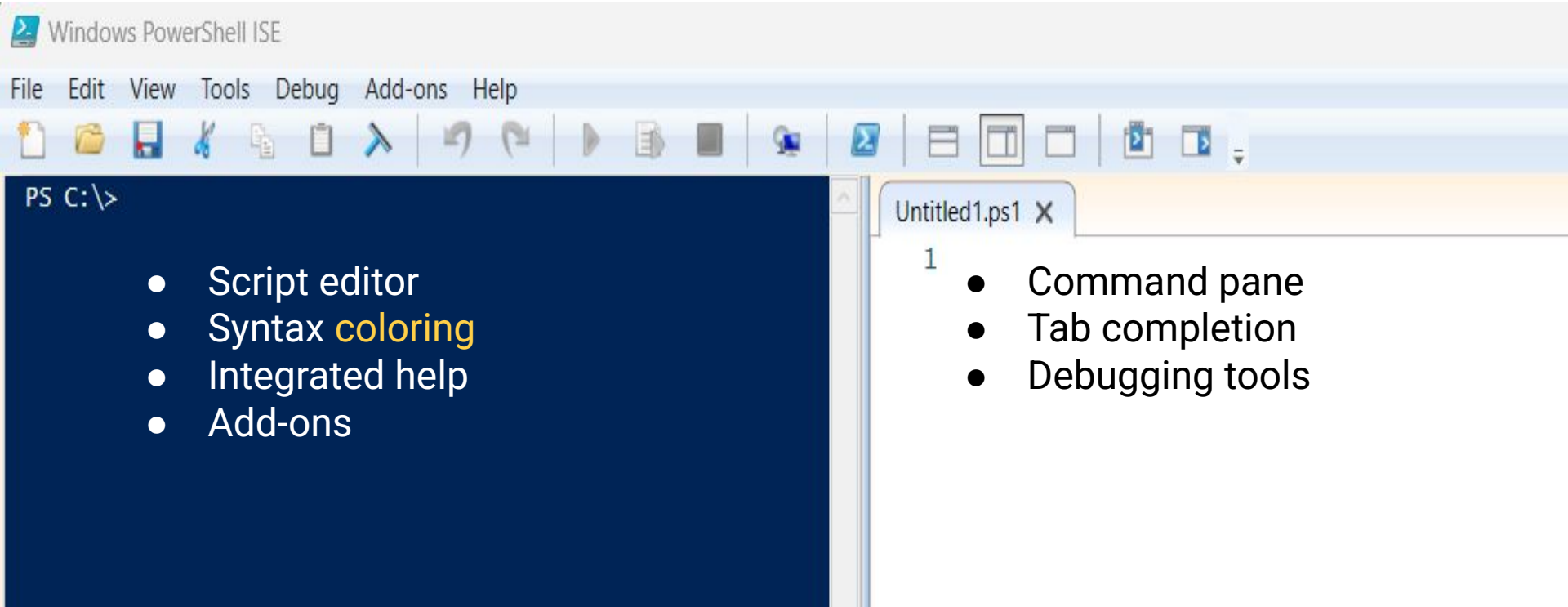
Based on the language used in the following commands, **what do you think they will do?**

```
PS C:\Users\administrator> Stop-Computer -Confirm
```

```
PS C:\Users\administrator> Stop-Computer -Force
```

PowerShell Integrated Scripting Environment (ISE)

Graphical host application for PowerShell





We can use PowerShell
piping for operations that
CMD doesn't support.

PowerShell Piping for CMD's Unsupported Operations

To find the file sizes of subdirectories in C:\Windows\System in CMD, we must use a complex batch file consisting of the following:

```
@echo off
set size=0
for /r %%x in (System\*) do set /a size+=%%~zx
echo %size% Bytes
```

PowerShell can do this with a simple pipe (|).

```
dir C:\Windows\System -Recurse | Measure-Object -Sum Length
```

Grabs all the current
directory and subdirectory
contents.

The output is piped into this
command, which measures
files.

PowerShell Piping for CMD's Unsupported Operations

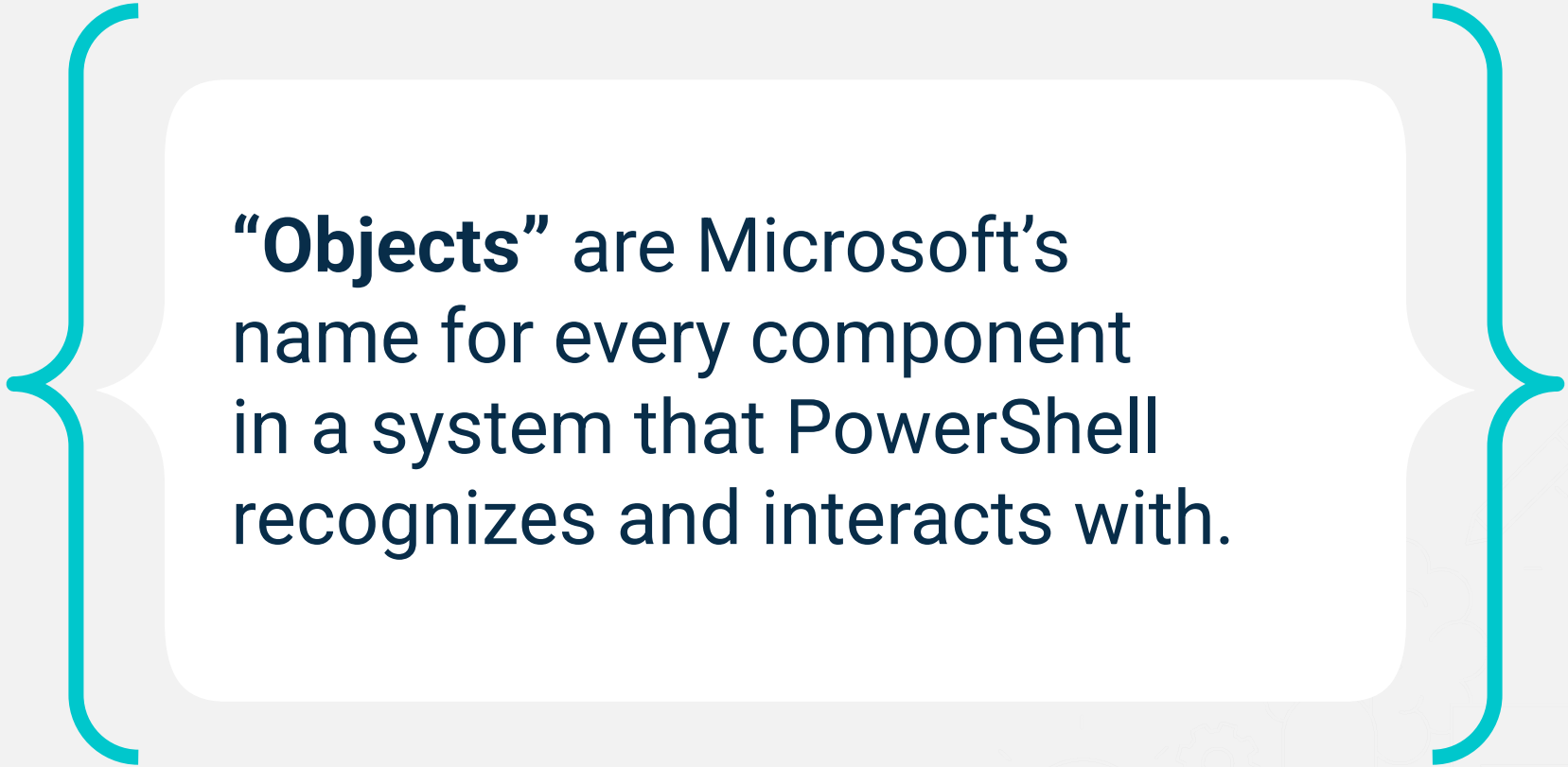
To find the file sizes of subdirectories in C:\Windows\System in CMD, we must use a complex batch file consisting of the following:

```
@echo off
set size=0
for /r %%x in (System\*) do set /a size+=%%~zx
echo %size% Bytes
```

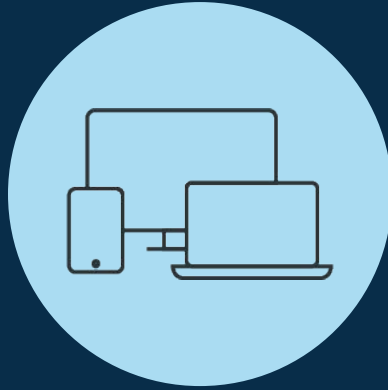
PowerShell can do this with a simple pipe (|).

```
dir C:\Windows\System -Recurse | Measure-Object
```

Objects are a very important PowerShell concept. We'll look at them next.



“Objects” are Microsoft’s
name for every component
in a system that PowerShell
recognizes and interacts with.



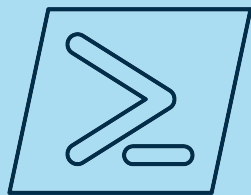
Instructor Demonstration

PowerShell and Objects Demo



Questions?





Intro to **PowerShell**



Demonstration Scenario

Let's continue our scenario from the previous lesson

Throughout today's lesson, we will be working with PowerShell to complete the following activities for our CIO:

- Navigate Windows with the PowerShell command line.
 - Create logs with PowerShell.
 - Create scripts with PowerShell.





PowerShell Syntax

cmdlets

- PowerShell uses commands known as **cmdlets** (“command-lets”)
- Combination of **verbs** and **nouns**

Measure-Object

Diagram illustrating the structure of a PowerShell cmdlet: **Measure-Object**. The word **Measure** is highlighted in yellow and **Object** is highlighted in teal. Brackets are placed under each word, indicating they are the verb and noun components respectively.



PowerShell Commands

cmdlet	Function	Equivalent Command
Set-Location	Changes to specified directory	cd
Get-ChildItem	Returns current directory contents	ls, dir
New-Item	Makes new file or directory	mkdir, touch
Remove-Item	Deletes file or directory	rm, rmdir
Get-Location	Retrieves path to current directory	pwd
Get-Content	Returns file content	cat, type
Copy-Item	Copies a file from one location to another	cp
Move-Item	Moves item from one location to another	mv
Write-Output	Prints output	echo
Get-Alias	Shows aliases from the current session	alias
Get-Help	Retrieves information about command	man
Get-Process	Retrieves processes running on machine	ps
Stop-Process	Stops specific process	kill
Get-Service	Retrieves list of services	service --status-all

Verbs-Nouns

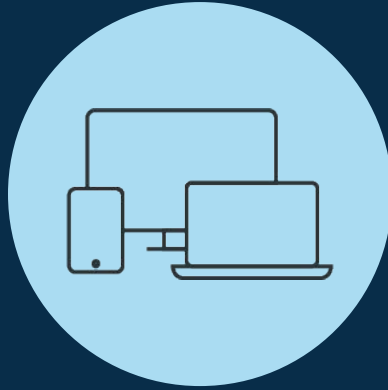
Consider the following scenario:

- User Alex left the company.
- We want to remove their user account from the system, but we want to keep the reports they were working on.
- We need to move the reports' files from their user desktop directory to a directory outside of the user.
- Along the way, we'll explore some other useful PowerShell commands.

We'll use the following commands:

`Set-Location` `Move-Item` `Get-ChildItem` `New-Item` `Remove-Item`



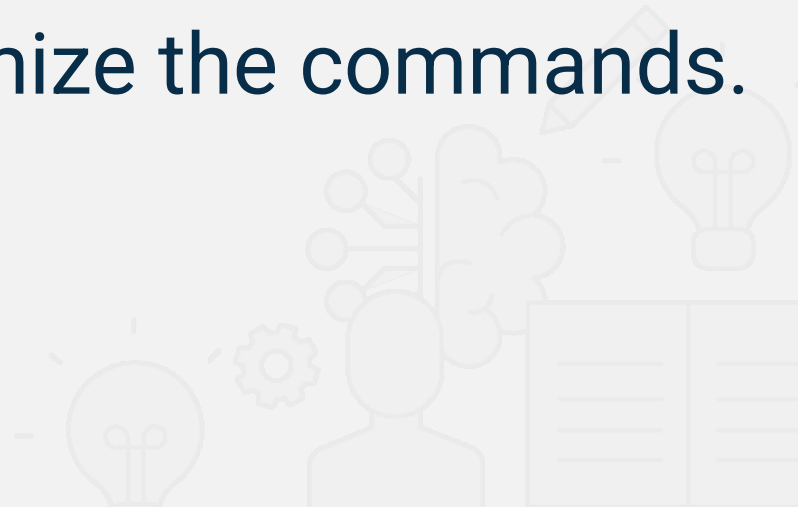


Instructor Demonstration

PowerShell Verb-Noun Demo



Now, we'll use **parameters** to customize the commands.



Parameters

Instead of using `New-Item` to create another file, we can add parameters to the `New-Item` command and create a directory, with a specific name and location:

We can use a `pipe` to retrieve only objects containing the word “system”:

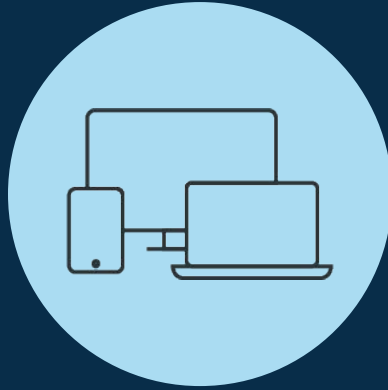
```
New-Item -Path "C:\" -Name "Logs" -ItemType "Directory"
```

-Path: Parameter specifying the location of this new directory.

-Name: Parameter specifying the directory's name.

-ItemType: Parameter specifying the type of item we want to create.

If we don't specify **"Directory"**, it will default to a file.



Instructor Demonstration

Parameter Examples



Activity:

Move and Create Directories

In this activity, you will work as a junior system administrator tasked with vetting a process to create Windows Event logs. These logs will later be integrated into a log management program known as a SIEM.

You'll run **PowerShell** commands to create, rename, and move items.

Suggested Time:
10 minutes



Time's up!
Let's review



Questions?

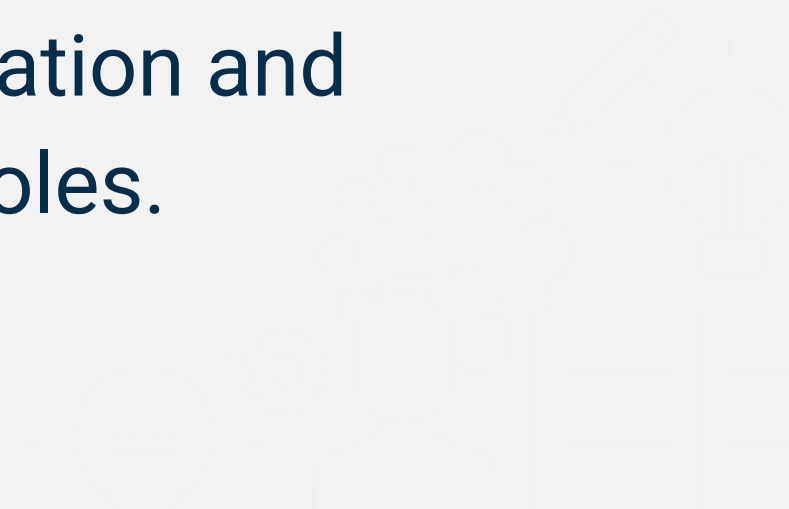




Generating Windows Event Log Files with **Parameters** and **Pipelines**



Chaining commands is important in system administration and security roles.



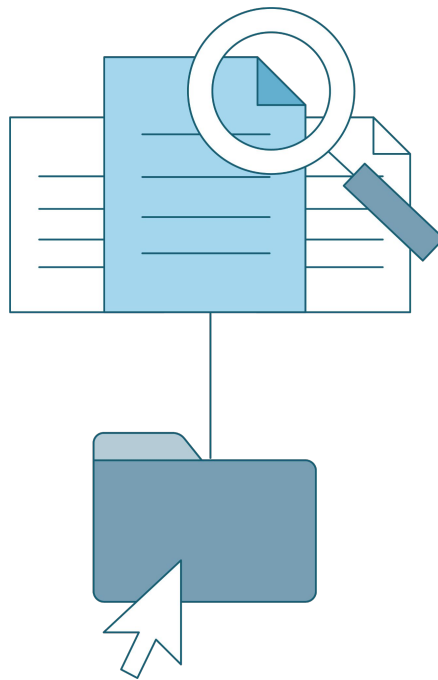


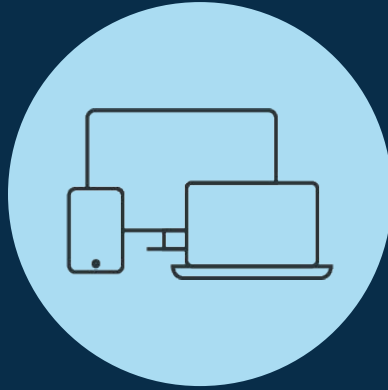
Demonstration Scenario

We will continue our role as a junior system administrator. Our CIO has asked us to retrieve logs and save them as **JSON** files in our newly created **C:\Logs** directory.

Chain `cmdlets` together that:

- Retrieve Windows logs from Windows Events.
- Transform the logs to the structured data format **JSON**.
- Output these files to the **C:\Logs** directory we made in the last activity.





Instructor Demonstration

PowerShell Parameters



Piping Logs to JavaScript Object Notation

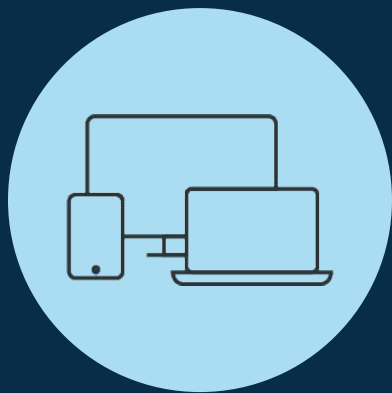
ConvertTo-Json

Now that we used parameters to get the logs we need, we will output them into a file that can be used later by log analysis applications.

This is where **pipelines** come in:

```
Get-WinEvent -LogName System -MaxEvents 10 | ConvertTo-Json
```

JSON (JavaScript Object Notation) structured data format can be easily read and manipulated by today's most common scripting and programming languages.



Instructor Demonstration

Piping Logs to **JSON**



Questions?





Activity:

Generating Windows Event Log Files with Parameters and Pipelines

In this activity, you will play the role of a Windows system administrator.

- You've been asked to create and save logs to the directory `C:\Logs`.
- These logs can then be sent to security analysts for examination in a Security Information and Event Management System (SIEM), which we will explore in depth during a later lesson.

Suggested Time:
20 Minutes



Break
15 mins



Time's up!
Let's review



Questions?



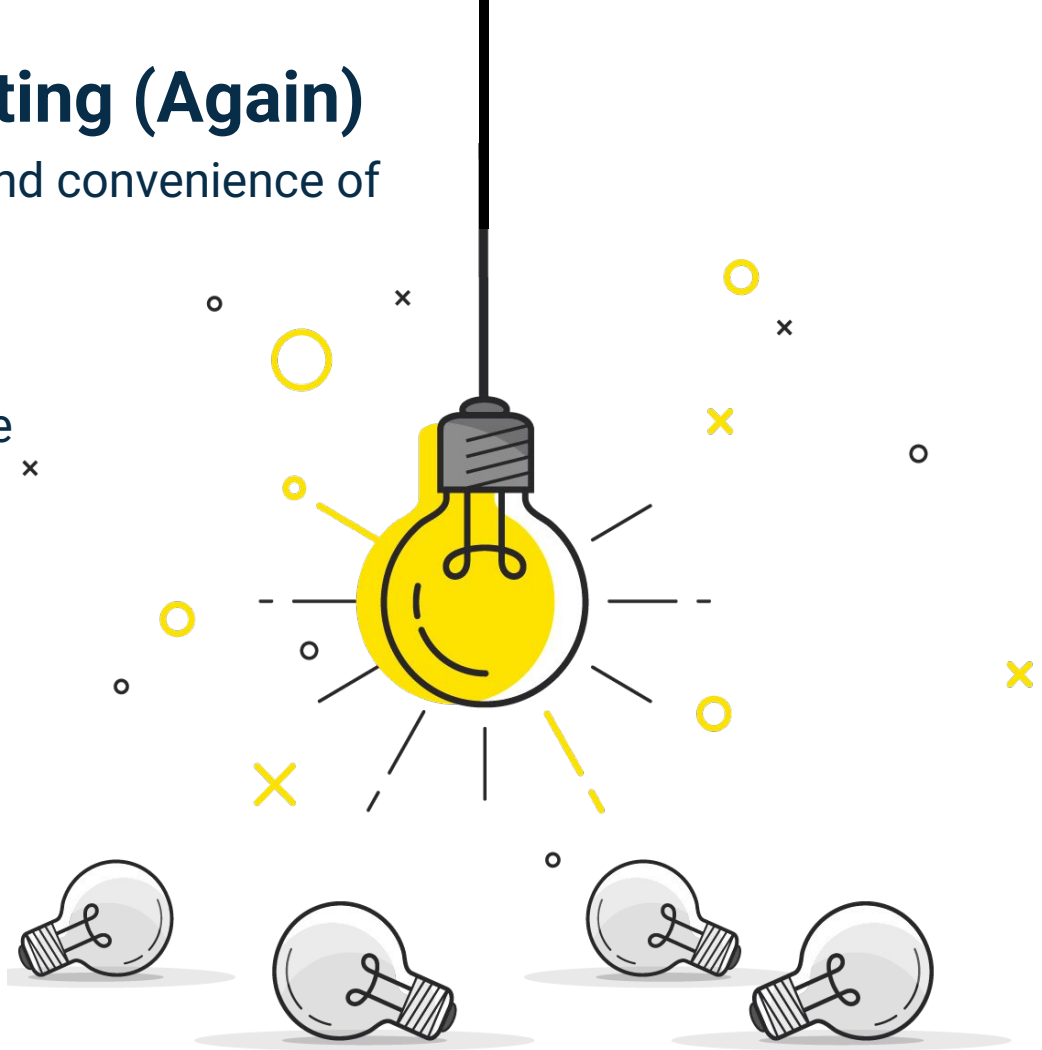


Scripting with PowerShell

The Importance of Scripting (Again)

We've emphasized the importance and convenience of scripting in our past sysadmin units.

- Scripts allow sysadmins and security professionals to automate and execute basic to advanced procedures and operations.
- Scripts can be used for everything from setting basic firewall rules to standing up entire cloud virtual machine environments with networking, storage, and users.





The Importance of Scripting (Again)



Scripting allows sysadmins and security professionals the ability to execute basic and advanced procedures:

- Firewall rules
- Cloud management
- Networking
- Storage
- Users



Scripting with PowerShell

Scripting commands in sequence

For example, you were asked to set up Windows workstations for users in the accounting department. You could create a script to:

01

Pull sensitive files to a specified directory

02

Download AppLocker

03

Deploy application policies for AppLocker, restricting user access for only people in the **accounting** group

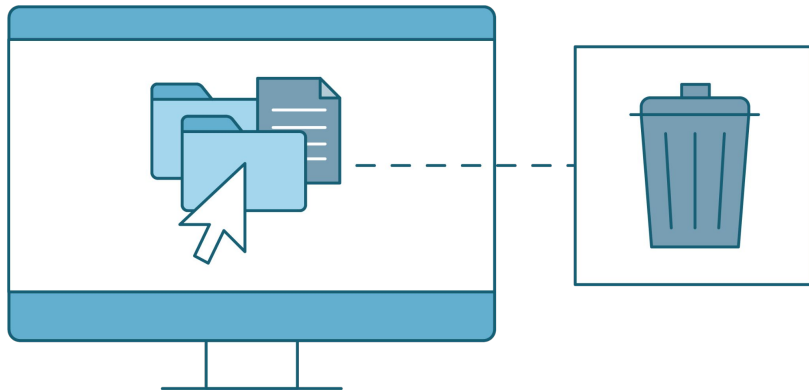


Demonstration Scenario

Scripting the Removal of Unnecessary Packages

In the upcoming demo, we will create a script in Git-Bash **Nano** to remove Microsoft Skype.

- This Windows workstation has installed a bunch of applications that are largely unnecessary and **potential vectors of attack**.
- These settings include telemetry tracking and advertising IDs, and default installed applications, including Skype.
- We want to **remove** these applications to reduce the attack surface area for this workstation.





Demonstration Scenario

By default, PowerShell does not allow scripts to run.



This is to prevent malware, attackers, and misconfigured scripts compromising a system.

Our systems have been configured to allow scripts to run, allowing automation.



Demonstration Scenario

In the demo, we've been tasked to:



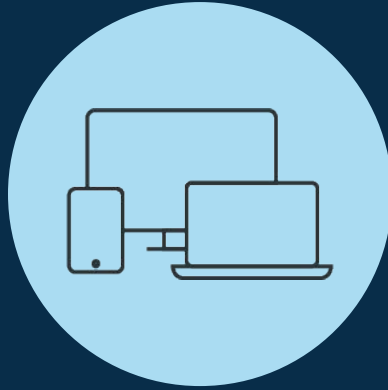
**Create a
PowerShell
script file and
execute it**



**Use a cmdlet
to import
various items**



**Use a foreach
loop**



Instructor Demonstration

Creating and Executing a PowerShell Script to Remove Applications

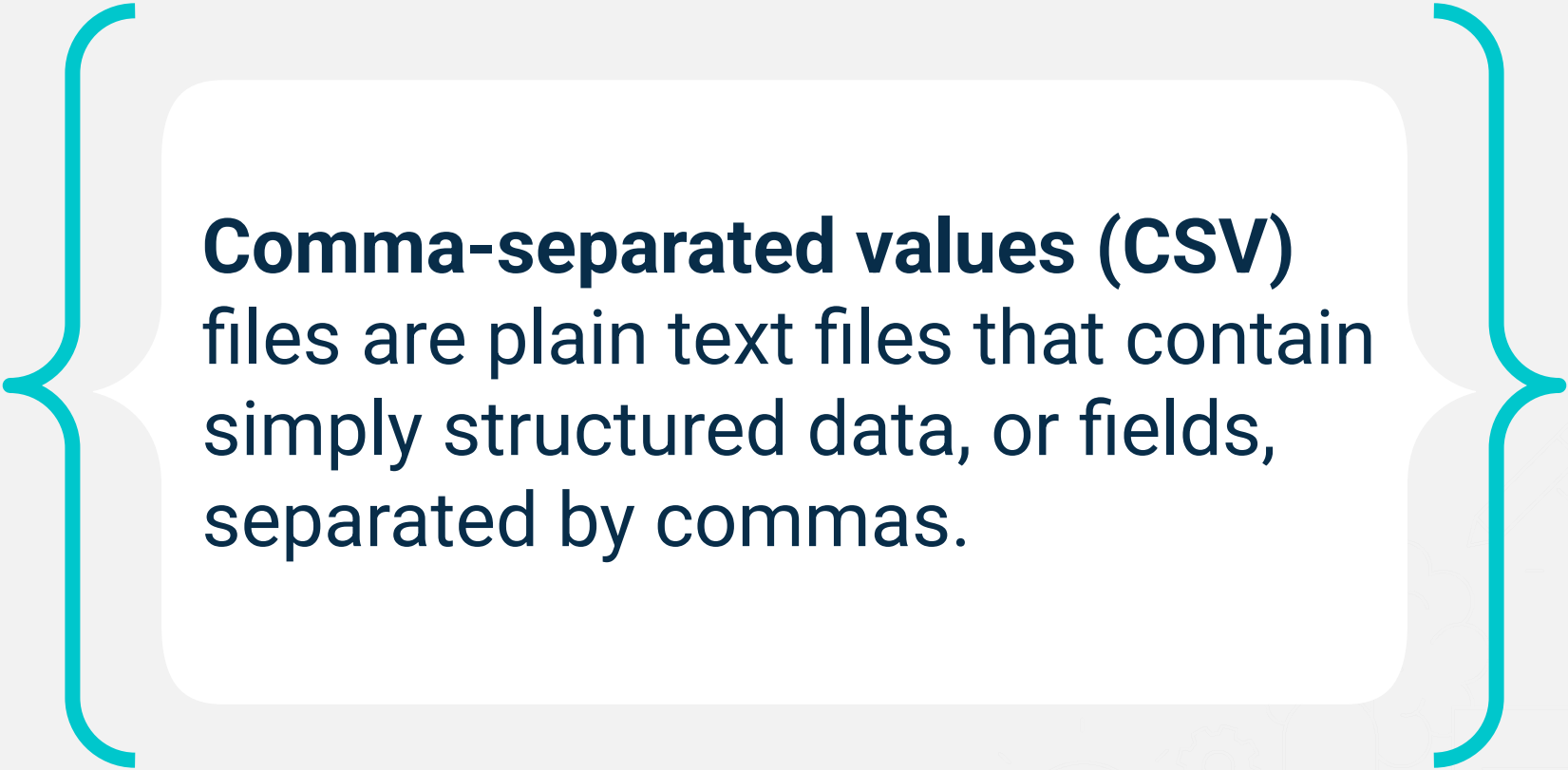
PowerShell Scripts and Removing Skype

We've just deleted a useless app from our machine.

While we could remove the Windows Store apps one by one, it would be more efficient to **create a script** that loops through a list of the apps and uninstalls them all at once.

More specifically, we can loop through **comma-separated values (CSV)** files.





Comma-separated values (CSV)
files are plain text files that contain
simply structured data, or fields,
separated by commas.



CSV Files

CSV files contain structured lines of information made up of rows of data.

Top line contains the **header**, describing each field.

```
Fruit,Vegetable
```

```
"Apple","Carrot"
```

```
"Banana","Broccoli"
```

```
"Orange","Spinach"
```

```
"Grapes","Peas"
```

```
"Strawberry","Cucumber"
```

CSV Files

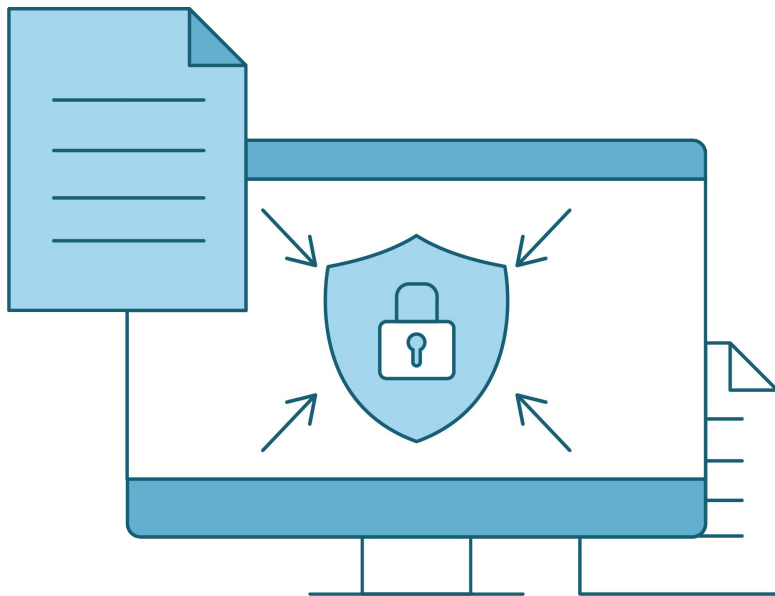
System Administrators and Security Professionals often use CSV files.

System Administrators

- Usernames
- Email addresses

Penetration Tester

- IP addresses
- Domains/subdomains



foreach Loops

We can use foreach loops to iterate through these files' items.

The foreach loop in **PowerShell** is similar to the for loop in Linux, but it is mainly used for looping through files or read-only structured data that you need to loop through.

Fruit	Vegetable
Apple	Carrot
Banana	Broccoli
Orange	Spinach

foreach Loops and CSV Files

Sysadmins and security professionals will use CSV and **foreach** loops together:

A system administrator may use a CSV file to maintain a list of employee email addresses and usernames.



They can use a **foreach** loop to loop through each item and change the passwords.

A penetration tester might have a list of IP addresses and corresponding domain and subdomain names to use in a test.

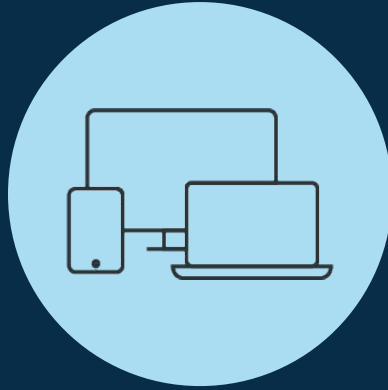


They can use a **foreach** loop to try out each password with a known username.

Now, We Will Continue our Demo Scenario

We will use the **Import-Csv** PowerShell cmdlet to source the CSV file `demo.csv` to run with a demo PowerShell script.

If the CSV file is constructed with the standard header and record setup, **Import-Csv** will prepare the fields to be called by PowerShell.



Instructor Demonstration

Import-CSV with a foreach Loop



Activity:

Removing Unnecessary Packages with PowerShell

In this activity, we will be continuing the role of junior system administrator.

- The CIO has tasked us with creating a PowerShell script that will get rid of these Windows bloat applications, and has provided a CSV file containing a list of these application's packages.

Suggested Time:
15 minutes



Time's up!
Let's review



Questions?





Next class

Active Directory Domain Services



Thank you